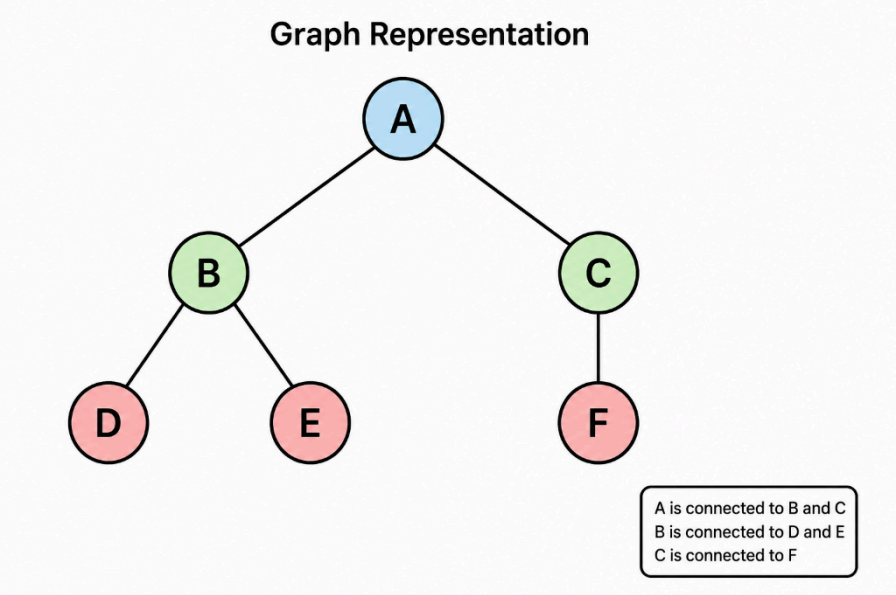
Practical No.:-01

Aim: Breadth First Search & Iterative Depth First Search



A. Implement the Breadth First Search algorithm to solve a given problem.

```
from collections import deque

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    print("BFS Traversal:")

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
```

```
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Run BFS
bfs(graph, 'A')
```

Output :

```
BFS Traversal:
A B C D E F
```

Explanation

1. Start from node **A**.
2. Visit all immediate neighbors (**B**, **C**).
3. Then visit neighbors of **B** (**D**, **E**).
4. Finally visit neighbor of **C** (**F**).

B. Implement the Iterative Depth First Search algorithm to solve the same problem.

```
# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    print("DFS Traversal:")

    while stack:
        node = stack.pop()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)

            # Push neighbors in reverse order
            for neighbor in reversed(graph[node]):
                if neighbor not in visited:
                    stack.append(neighbor)
```

```
# Run DFS
dfs_iterative(graph, 'A')
```

```
DFS Traversal:
A B D E C F
```

Step-by-Step Execution

Stack	**Visited Node**
A	A
B, C	B
D, E, C	D
E, C	E
C	C
F	F

C. Compare the performance and efficiency of both algorithms.

Performance Comparison:

Feature	**BFS**	**Iterative DFS**
Data Structure	Queue	Stack
Search Strategy	Level by level	Go as deep as possible
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V)	O(V)
Memory Usage	Higher	Lower
Shortest Path	Yes (unweighted graph)	No guarantee
Suitable For	Finding shortest paths	Exploring all nodes quickly

Where:

- **V** = Number of vertices
- **E** = Number of edges

Efficiency Analysis:

Breadth First Search (BFS)

Advantages

- Finds the shortest path in an unweighted graph.
- Explores nodes systematically level by level.
- Useful in routing, networking, and pathfinding problems.

Disadvantages

- Requires more memory because all neighboring nodes at a level are stored in the queue.
- Can be slower in very large graphs due to memory overhead.

Iterative Depth First Search (DFS)

Advantages

- Uses less memory because only the current path is stored in the stack.
- Suitable for large graphs where memory is limited.
- Useful for maze solving, cycle detection, and graph traversal.

Disadvantages

- Does not guarantee the shortest path.
- May explore a long branch before finding the desired node.

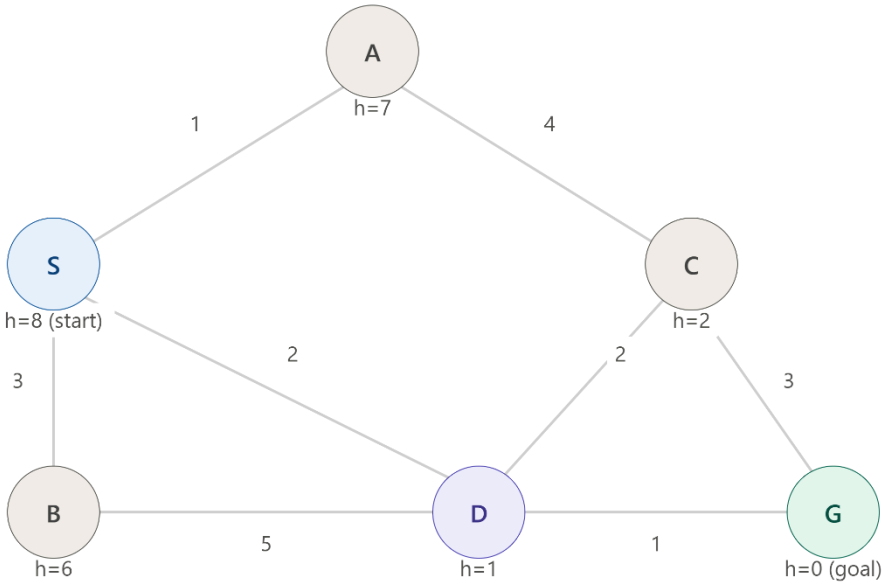
Conclusion

- Choose **BFS** when you need the **shortest path** or want to explore nodes level by level.
- Choose **Iterative DFS** when **memory efficiency** is important and finding the shortest path is not required.
- Both algorithms have the same theoretical time complexity **O(V + E)**, but **DFS is generally more memory-efficient**, while **BFS is more suitable for shortest-path problems**.

Practical No : 2

A* Search and Recursive Best-First Search

Problem Statement:



A. Implement the A* Search algorithm for solving a pathfinding problem.

Algorithm: A Search Algorithm*

1. Start the algorithm.
2. Define the graph with nodes and edge costs.
3. Define the heuristic values h(n) for all nodes.
4. Create a priority queue (pq) and insert the start node with:

   f = h(Start)

   g = 0

   path = [Start]
5. Repeat while the priority queue is not empty:

   Remove the node with the smallest f value.

   If the current node is the Goal (G):

   - Print the shortest path.
   - Print the total cost.
   - Stop the algorithm.

   Otherwise, visit all neighboring nodes.

For each neighbor:

- Calculate the new cost:

  g=current cost+edge cost g = \text{current cost} + \text{edge cost}g=current cost+edge cost

- Calculate:

  f=g+h(neighbor)f = g + h(\text{neighbor})f=g+h(neighbor)

- Insert the neighbor into the priority queue along with its updated path.

6. End the algorithm.

Code:

```
import heapq

graph = {
    'S': [('A',1), ('B',3), ('D',2)],
    'A': [('C',4)],
    'B': [('D',5)],
    'C': [('G',3)],
    'D': [('G',1)],
    'G': []
}

h = {'S':8, 'A':7, 'B':6, 'C':2, 'D':1, 'G':0}

start_node= 'B'

pq = [(h[start_node], 0, start_node, [start_node])] #(f, g, node, path)

while pq:
    f, g, node, path = heapq.heappop(pq) # Removes and returns the smallest element.

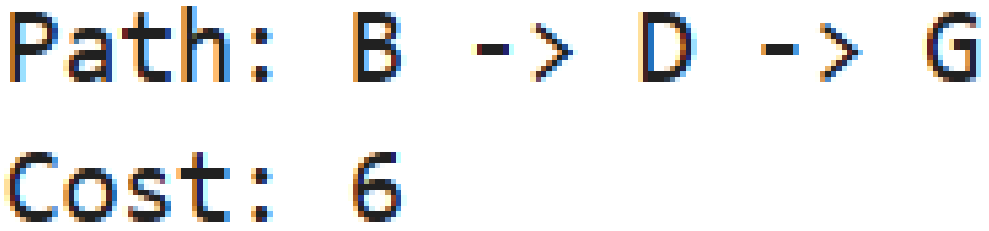
    if node == 'G':
        print("Path:", " -> ".join(path))
        print("Cost:", g)

        break

#This part of the code expands the neighboring nodes of the current node and adds them to the priority queue.

    for nxt, cost in graph[node]:
        heapq.heappush(pq, (g + cost + h[nxt], g + cost, nxt, path + [nxt]))
```

Output:



B. Implement the Recursive Best-First Search algorithm for the same problem.

Algorithm (Stepwise)

1. Start from the **Start node**.
2. Calculate **f(n) = g(n) + h(n)**.
3. Expand the current node and generate all its neighboring nodes.
4. Calculate the **f-value** for each neighbor.
5. Select the neighbor with the **smallest f-value** (best node).
6. Recursively explore the best node.
7. If the best node is the **Goal**, return the path.
8. If the best path exceeds the current **f-limit**, backtrack and explore the next best node.
9. Repeat until the Goal is found.
10. Stop.

Code

```
graph = {
   'S': [('A',1), ('B',3), ('D',2)],
   'A': [('C',4)],
   'B': [('D',5)],
   'C': [('G',3)],
   'D': [('G',1)],
   'G': []
}

h = {'S':8, 'A':7, 'B':6, 'C':2, 'D':1, 'G':0}

def rbfs(node, goal, cost, path):
   if node == goal:
```

```
        print("Path:", " -> ".join(path))
        print("Cost:", cost)
        return True

    if not graph[node]:
        return False

    graph[node].sort(key=lambda x: cost + x[1] + h[x[0]])

    for nxt, c in graph[node]:
        if rbfs(nxt, goal, cost + c, path + [nxt]):
            return True

    return False

rbfs('S', 'G', 0, ['S'])
```

Output:

```
Path: S -> D -> G
Cost: 3

True
```

C. Compare the performance and effectiveness of both algorithms.

Feature	A* Search	Recursive Best-First Search (RBFS)
Search Strategy	Best-First Search using a Priority Queue	Best-First Search using Recursion
Data Structure	Priority Queue (heapq)	Recursive Function
Memory Usage	High (stores all generated nodes)	Low (stores only the current path)
Time Complexity	**$O(b^d)$** (Worst Case)	**$O(b^d)$** (Worst Case)
Space Complexity	**$O(b^d)$**	**$O(bd)$**
Speed	Faster	Slightly slower due to backtracking
Backtracking	No	Yes
Optimal Solution	Yes (with admissible heuristic)	Yes (with admissible heuristic)
Suitable For	Small to medium problems with sufficient memory	Large problems where memory is limited

Performance Comparison

- A* is generally faster because it stores all generated nodes in a priority queue and always expands the node with the smallest f-value.
- RBFS may revisit nodes due to recursive backtracking, making it slower than A* in some cases.

Effectiveness Comparison

- A* is more effective when enough memory is available, as it quickly finds the optimal path.
- RBFS is more effective for memory-constrained systems because it uses much less memory while still finding the optimal path.

Conclusion

- Use A* when speed is important and sufficient memory is available.
- Use RBFS when memory is limited, even though it may take slightly longer because of recursive backtracking. Both algorithms find the optimal path when the heuristic function is admissible.

Practical No 3

Decision Tree Learning

A. Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset

Algorithm :

1. Import required libraries.
2. Load the dataset.
3. Convert categorical data into numeric form.
4. Separate features (X) and target (y).
5. Create the Decision Tree classifier.
6. Train the model using fit().
7. Predict the output for a sample record.
8. Display the prediction.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# Load dataset
data = pd.read_csv("play_tennis.csv")

# Convert text data into numbers
data = pd.get_dummies(data)

# Input (features) and Output (target)
X = data.drop("PlayTennis_Yes", axis=1)
y = data["PlayTennis_Yes"]

# Create and train Decision Tree
model = DecisionTreeClassifier()
model.fit(X, y)

# Predict using the first record
prediction = model.predict([X.iloc[0]])

print("Prediction:", "Yes" if prediction[0] == 1 else "No")
```

Explanation

1. pd.read_csv() → Loads the dataset.

2. pd.get_dummies() → Converts categorical values (Sunny, Rain, Hot, etc.) into numeric values.
3. X → Input features.
4. y → Target column (PlayTennis).
5. DecisionTreeClassifier() → Creates the Decision Tree model.
6. fit() → Trains the model.
7. predict() → Predicts the class for a new record.

B. Evaluate the accuracy and effectiveness of the decision tree on test data.

```
from sklearn.model_selection import  train_test_split
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report


# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create and train Decision Tree
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)

# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))


accuracy = accuracy_score(y_test, y) * 100
print("Accuracy: {:.2f}%".format(accuracy))
```

Output:

```
Accuracy: 1.0

Confusion Matrix:
[[3 0]
 [0 3]]

Classification Report:
              precision    recall  f1-score   support

       False       1.00      1.00      1.00         3
        True       1.00      1.00      1.00         3

    accuracy                           1.00         6
   macro avg       1.00      1.00      1.00         6
weighted avg       1.00      1.00      1.00         6

Accuracy: 100.00%
```

C.Visualize and interpret the generated decision tree.

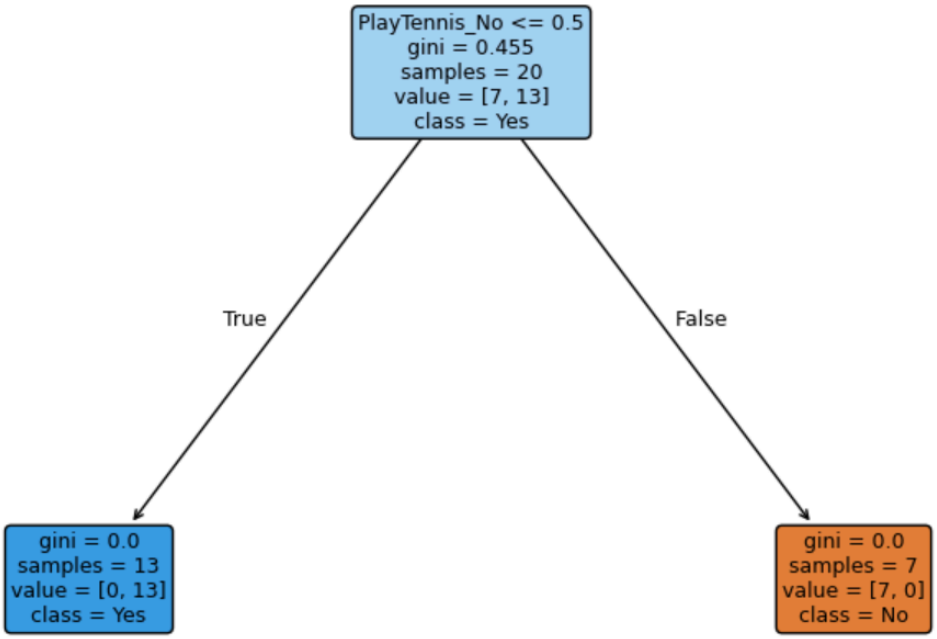
```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Visualize the Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No", "Yes"],
    filled=True,
    rounded=True,
    fontsize=9
)

plt.title("Decision Tree for Play Tennis Dataset")
plt.show()
```

Output:

Decision Tree for Play Tennis Dataset



Practical No 4:

Feed Forward Backpropagation Neural Network

A. Implement the Feed Forward Backpropagation algorithm to train a neural network.

```
from sklearn.datasets import load_iris
from sklearn.neural_network import MLPClassifier

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Create Neural Network
model = MLPClassifier(hidden_layer_sizes=(5),
                      max_iter=500,
                      random_state=1)

# Train Model
```

```
model.fit(X, y)

print("Training Completed Successfully")
```

Explanation

- Load the Iris dataset.
- Create a Feed Forward Neural Network having one hidden layer with 5 neurons.
- Train the network using the Backpropagation algorithm.
- Display a success message.

Output

Training Completed Successfully

Result

The Feed Forward Backpropagation Neural Network was implemented successfully.

B. Use a given dataset to train the neural network for a specific task.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)

# Create Model
model = MLPClassifier(hidden_layer_sizes=(5),
                      max_iter=500,
                      random_state=1)

# Train Model
model.fit(X_train, y_train)
```

```
print("Model Trained Successfully")
```

Explanation

- Load the Iris dataset.
- Split data into 70% training and 30% testing.
- Train the Feed Forward Neural Network.
- The network learns using the Backpropagation algorithm.

Output

Model Trained Successfully

C. Evaluate the performance of the trained network on test data.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)

# Train Model
model = MLPClassifier(hidden_layer_sizes=(5),
                      max_iter=500,
                      random_state=1)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy =", round(accuracy*100,2), "%")
```

Output :

Accuracy = 71.11 %

Explanation

- Test data is given to the trained model.
- The model predicts the output.
- Accuracy is calculated using:

  Accuracy= (Correct Predictions/Total Predictions) * 100

Practical No 5

Support Vector Machines (SVM)

A. Implement the SVM algorithm for binary classification.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv("SVM_Binary_Classification_Dataset.csv")

# Features and target
X = df[["Hours_Studied", "Attendance"]]
y = df["Result"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Train SVM
model = SVC(kernel="linear")
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual:", y_test.values)
print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
```

B. Train an SVM model using a given dataset and optimize its parameters

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset
```

```
df = pd.read_csv("SVM_Binary_Classification_Dataset.csv")

X = df[["Hours_Studied", "Attendance"]]
y = df["Result"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# SVM model
model = SVC()

# Parameters to optimize
params = {
    "C": [0.1, 1, 10],
    "kernel": ["linear", "rbf"],
    "gamma": ["scale", "auto"]
}

# Find best parameters
grid = GridSearchCV(model, params, cv=3)
grid.fit(X_train, y_train)

# Test best model
y_pred = grid.predict(X_test)

print("Best Parameters:", grid.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
```

C. Evaluate the performance of the SVM model on test data and analyze the results.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
df = pd.read_csv("SVM_Binary_Classification_Dataset.csv")

X = df[["Hours_Studied", "Attendance"]]
y = df["Result"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Train SVM
model = SVC(kernel="linear")
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Evaluate model
print("Actual:", y_test.values)
print("Predicted:", y_pred)
print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

1. Accuracy shows the percentage of correctly classified students.
2. The confusion matrix shows correct and incorrect predictions for Pass and Fail.
3. Precision indicates how many predicted classes were actually correct.
4. Recall indicates how well the model identifies each class.
5. F1-score gives a balance between precision and recall.

Practical No 6

Adaboost Ensemble Learning

A. Implement the Adaboost algorithm to create an ensemble of weak classifiers.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Weak classifier
weak_model = DecisionTreeClassifier(max_depth=1)

# AdaBoost ensemble
model = AdaBoostClassifier(
    estimator=weak_model,
    n_estimators=50,
    random_state=42
)
 #########----------(6B)----------############
# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Predicted values:", y_pred)
```

1. AdaBoost (Adaptive Boosting) combines multiple weak classifiers to create a strong classifier.
2. Here, a Decision Tree with depth = 1 is used as the weak classifier.
3. `n_estimators=50` creates 50 weak classifiers sequentially.
4. Each new classifier focuses more on the samples that were incorrectly classified by previous classifiers.
5. Finally, all weak classifiers are combined to make the final prediction.
6. Accuracy shows how well the AdaBoost model performs on the test data.

Practical No 7

Aim: Naive Bayes' Classifier

● Implement the Naive Bayes algorithm for classification.

● Trin a Naive Bayes' model using a given dataset and calculate class probabilities.

●Evaluate the accuracy of the model on test data and analyze the results.

This practical requires a local file named disease.csv. You must create this file in the same directory Install all require libreries

Install all require libreries

Practical No.:-07

Aim: Naive Bayes' Classifier

- Implement the Naive Bayes algorithm for classification.
- Trin a Naive Bayes' model using a given dataset and calculate class probabilities.
- Evaluate the accuracy of the model on test data and analyze the results.

This practical requires a local file named **disease.csv**. You must **create this file** in the same directory

Install all require libreries

[1]:
```
pip install pandas scikit-learn matplotlib seaborn
```

Load & Preprocess Dataset

[2]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, CategoricalNB, GaussianNB
from sklearn.metrics import accuracy_score
import seaborn as sns
```

[4]:
```
df = pd.read_csv('diseases.csv')
```

[5]:
```
df.head(11)
```

[7]:

	Sore Throat	Fever	Swollen Glands	Congestion	Headache	Diagnosis
0	No	Yes	No	Yes	Yes	Allergy
1	No	No	Yes	Yes	Yes	Allergy
2	No	No	No	No	Yes	Allergy
3	No	Yes	No	No	No	Strep throat
4	Yes	Yes	No	Yes	No	Cold
5	Yes	Yes	Yes	Yes	Yes	Strep throat
6	No	No	No	No	Yes	Cold
7	No	No	Yes	No	No	Cold
8	Yes	No	No	Yes	Yes	Strep throat
9	No	Yes	Yes	Yes	No	Allergy
10	Yes	No	No	No	No	Strep throat

[8]:
```
df.tail()
```

[8]:

	Sore Throat	Fever	Swollen Glands	Congestion	Headache	Diagnosis
25	Yes	Yes	No	No	Yes	Allergy
26	No	No	No	No	No	Strep throat
27	Yes	Yes	No	No	Yes	Strep throat
28	Yes	No	Yes	Yes	Yes	Cold
29	Yes	Yes	No	Yes	No	Allergy

[9]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   Sore Throat     30 non-null     object
 1   Fever           30 non-null     object
 2   Swollen Glands  30 non-null     object
 3   Congestion      30 non-null     object
 4   Headache        30 non-null     object
 5   Diagnosis       30 non-null     object
dtypes: object(6)
memory usage: 1.5+ KB
```

[10]:
```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Sore Throat'] = le.fit_transform(df['Sore Throat'])
df['Fever'] = le.fit_transform(df['Fever'])
df['Swollen Glands'] = le.fit_transform(df['Swollen Glands'])
df['Congestion'] = le.fit_transform(df['Congestion'])
df['Headache'] = le.fit_transform(df['Headache'])
df['Diagnosis'] = le.fit_transform(df['Diagnosis'])
```

[11]:
```
df.info()
```

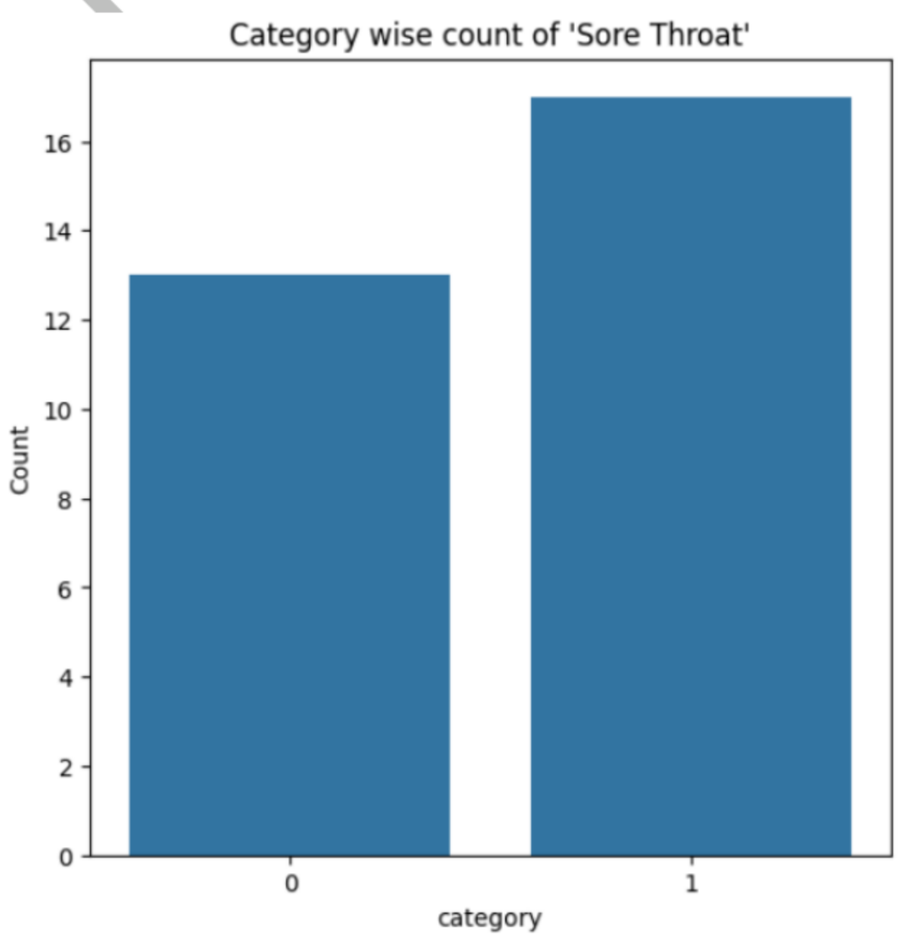
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Sore Throat     30 non-null     int32
 1   Fever           30 non-null     int32
 2   Swollen Glands  30 non-null     int32
 3   Congestion      30 non-null     int32
 4   Headache        30 non-null     int32
 5   Diagnosis       30 non-null     int32
dtypes: int32(6)
memory usage: 852.0 bytes
```

[12]:
```
df.head(11)
```

[12]:

	Sore Throat	Fever	Swollen Glands	Congestion	Headache	Diagnosis
0	0	1	0	1	1	0
1	0	0	1	1	1	0
2	0	0	0	0	1	0
3	0	1	0	0	0	2
4	1	1	0	1	0	1
5	1	1	1	1	1	2
6	0	0	0	0	1	1
7	0	0	1	0	0	1
8	1	0	0	1	1	2
9	0	1	1	1	0	0
10	1	0	0	0	0	2

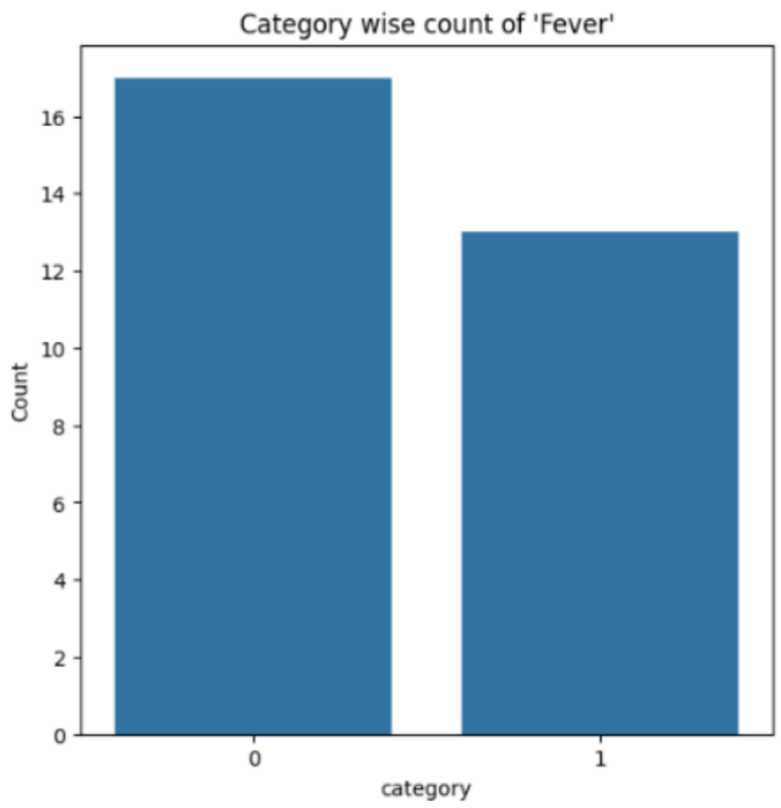
[13]:
```
# In [13]
fig, ax = plt.subplots(figsize=(6,6))
sns.countplot(x=df['Sore Throat'], data=df)
plt.title("Category wise count of 'Sore Throat'")
plt.xlabel("category")
plt.ylabel("Count")
plt.show()
```



SDSM

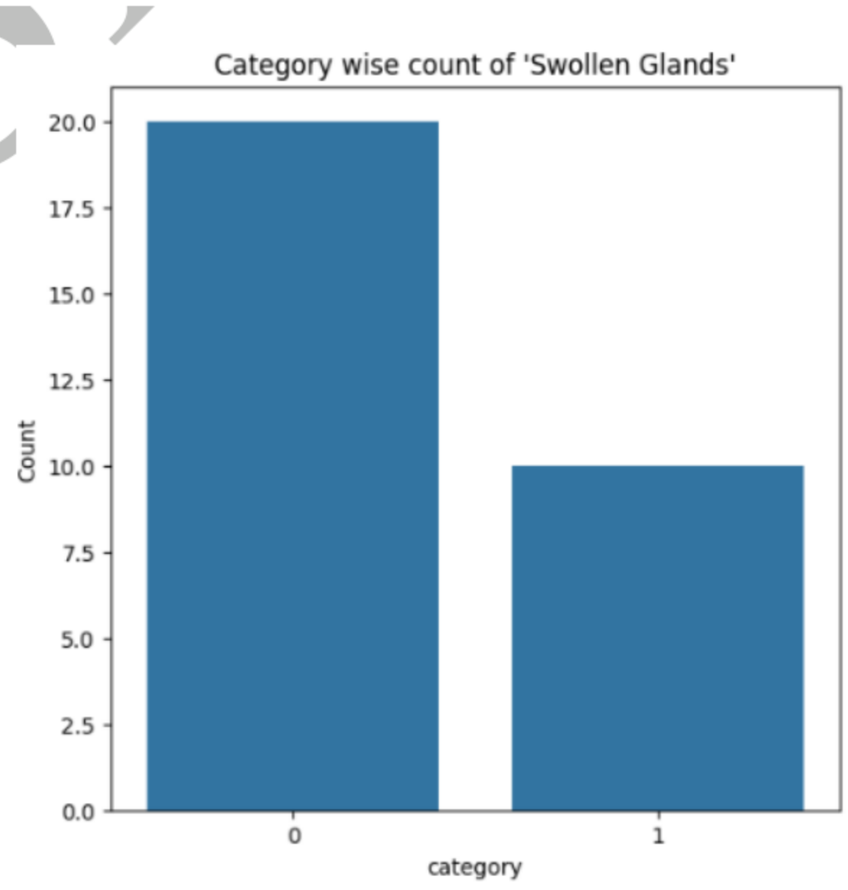
[17]:

```
fig, ax = plt.subplots(figsize=(6,6))
sns.countplot(x=df['Fever'], data=df)
plt.title("Category wise count of 'Fever'")
plt.xlabel("category")
plt.ylabel("Count")
plt.show()
```



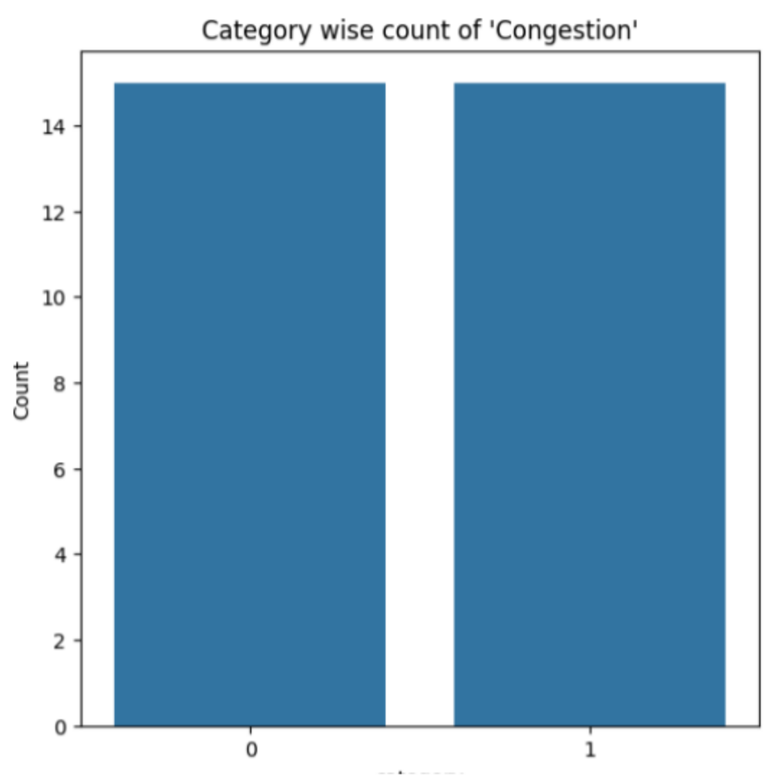
[18]:

```
fig, ax = plt.subplots(figsize=(6,6))
sns.countplot(x=df['Swollen Glands'], data=df)
plt.title("Category wise count of 'Swollen Glands'")
plt.xlabel("category")
plt.ylabel("Count")
plt.show()
```



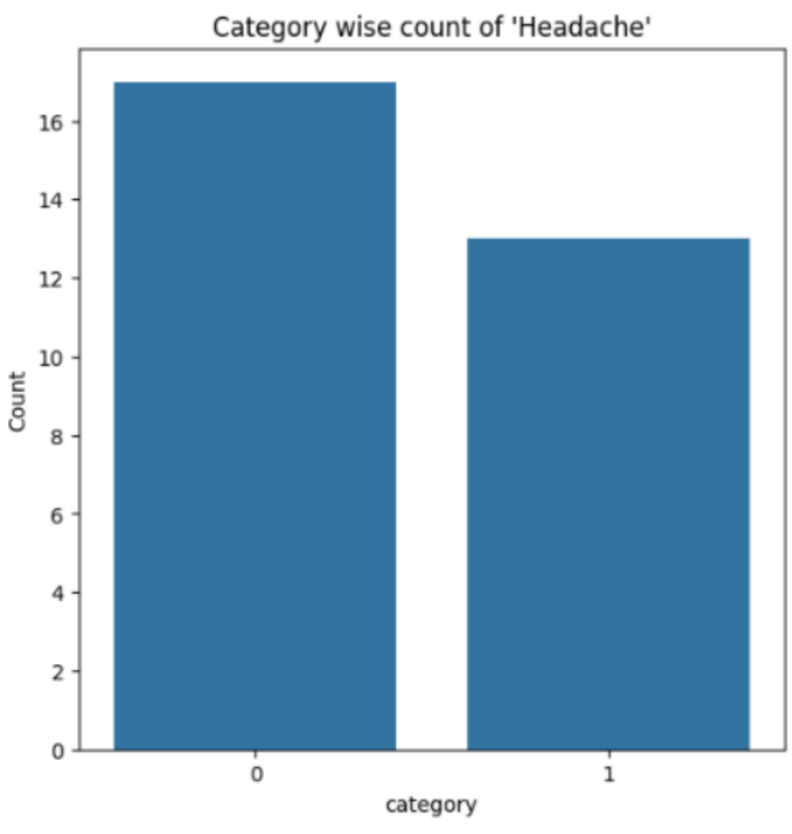
[19]:

```
fig, ax = plt.subplots(figsize=(6,6))
sns.countplot(x=df['Congestion'], data=df)
plt.title("Category wise count of 'Congestion'")
plt.xlabel("category")
plt.ylabel("Count")
plt.show()
```



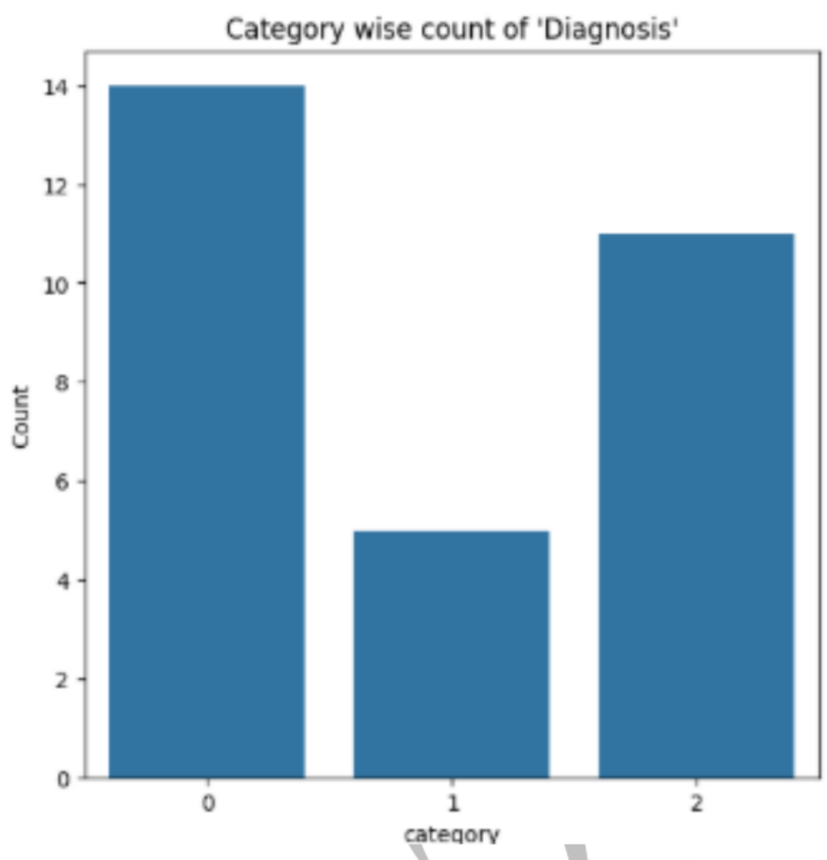
[20]:

```
fig, ax = plt.subplots(figsize=(6,6))
sns.countplot(x=df['Headache'], data=df)
plt.title("Category wise count of 'Headache'")
plt.xlabel("category")
plt.ylabel("Count")
plt.show()
```



[21]:

```
fig, ax = plt.subplots(figsize=(6,6))
sns.countplot(x=df['Diagnosis'], data=df)
plt.title("Category wise count of 'Diagnosis'")
plt.xlabel("category")
plt.ylabel("Count")
plt.show()
```



[22]:

```
X = df.drop('Diagnosis', axis=1)
y = df['Diagnosis']
```

Implement Naive Bayes

[23]:

```
#Training algorithm
classifier = MultinomialNB()
classifier.fit(X, y)
```

[23]:

▸ MultinomialNB

[24]:

```
#Training algorithm
classifier = CategoricalNB()
classifier.fit(X, y)
```

[24]:

▸ CategoricalNB

[25]:

```
#Training algorithm
classifier = GaussianNB()
classifier.fit(X, y)
```

[25]:

▸ GaussianNB

Train/Test Split & Training

[26]:
```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix, precisio
```

[26]

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import classification_report, accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

[27]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Evaluate Model

[28]:
```
classifier = MultinomialNB()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print("confusion matrix\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='macro'))
print("Recall:", recall_score(y_test, y_pred, average='macro'))
print("F1 score:", f1_score(y_test, y_pred, average='macro'))
print("Classification report:\n", classification_report(y_test, y_pred))
```

Output:-

```
confusion matrix
 [[2 0]
 [4 0]]
Accuracy: 0.3333333333333333
Precision: 0.16666666666666666
Recall: 0.5
F1 score: 0.25
Classification report:
                precision    recall  f1-score   support

           0       0.33      1.00      0.50         2
           2       0.00      0.00      0.00         4

    accuracy                           0.33         6
   macro avg       0.17      0.50      0.25         6
weighted avg       0.11      0.33      0.17         6
```

Practical No.:-08

Aim: **K-Nearest Neighbors (K-NN)**

- Implement the K-NN algorithm for classification or regression.
- Apply the K-NN algorithm to a given dataset and predict the class or value for test data.
- Evaluate the accuracy or error of the predictions and analyze the results.

Libraries to Install:-

[29]:

```
pip install numpy pandas matplotlib scikit-learn
```

This code uses the **Pima Indians Diabetes Dataset.** You need to download it and save it as **diabetes.csv** in the **same folder**

[1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.style.use('ggplot')
```

[2]:

```
df = pd.read_csv('diabetes.csv')
df.head()
```

[2]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

[3]:

```
df.shape
```

```
(768, 9)
```

[4]:

```
df.dtypes
```

[4]:

```
Pregnancies                 int64
Glucose                     int64
BloodPressure               int64
SkinThickness               int64
Insulin                     int64
BMI                       float64
DiabetesPedigreeFunction  float64
Age                         int64
Outcome                     int64
dtype: object
```

[5]:
```
X = df.drop('Outcome', axis=1).values
y = df['Outcome'].values
```

[6]:
```
from sklearn.model_selection import train_test_split
```

[7]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42, str
```

[7]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42, stratify=y)

[8]:

```
from sklearn.neighbors import KNeighborsClassifier

neighbors = np.arange(1, 9)
train_accuracy = np.empty(len(neighbors))
test_accuracy = np.empty(len(neighbors))

for i, k in enumerate(neighbors):
    # Setup a k-NN Classifier with k neighbors
    knn = KNeighborsClassifier(n_neighbors=k)

    # Fit the classifier to the training data
    knn.fit(X_train, y_train)

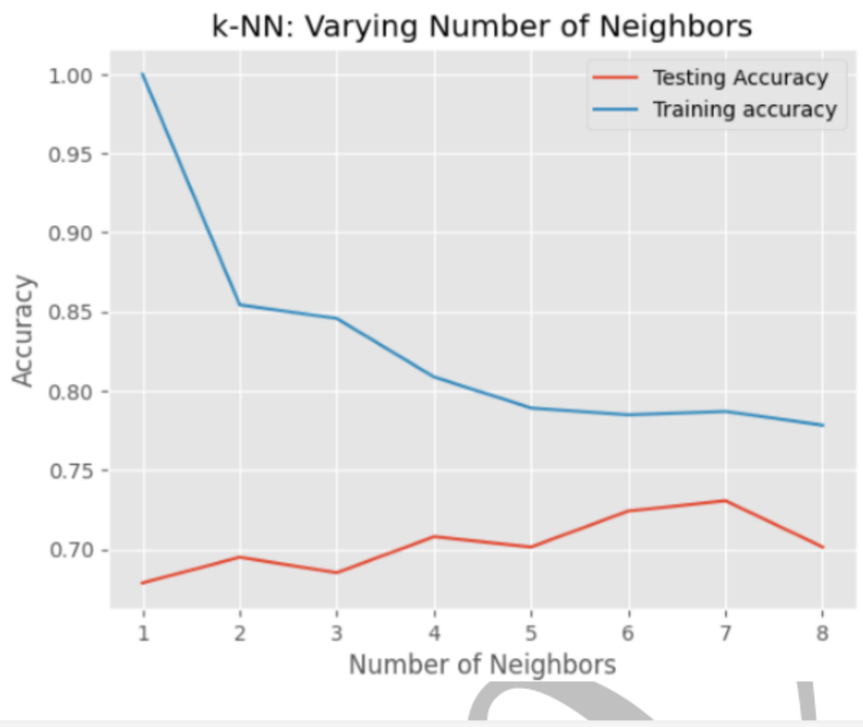
    # Compute accuracy on the training set
    train_accuracy[i] = knn.score(X_train, y_train)

    # Compute accuracy on the testing set
    test_accuracy[i] = knn.score(X_test, y_test)
```

Note:-The **train_accuracy** and **test_accuracy** arrays are now filled with values.

[9]:

```
plt.title('k-NN: Varying Number of Neighbors')
plt.plot(neighbors, test_accuracy, label='Testing Accuracy')
plt.plot(neighbors, train_accuracy, label='Training accuracy')
plt.legend()
plt.xlabel('Number of Neighbors')
plt.ylabel('Accuracy')
plt.show()
```



[10]:

```
# Setup a k-NN Classifier with 7 neighbors
knn = KNeighborsClassifier(n_neighbors=7)

# Fit the classifier to the training data
knn.fit(X_train, y_train)

# Get the accuracy of the model on the test data
knn.score(X_test, y_test)
```

[10]:

```
0.7305194805194806
```

[11]:

```
from sklearn.metrics import confusion_matrix

# Let's get the predictions for the test set
y_pred = knn.predict(X_test)

# Generate the confusion matrix
confusion_matrix(y_test, y_pred)
```

[11]:

```
array([[165,  36],
       [ 47,  60]], dtype=int64)
```

[12]:

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.78      0.82      0.80       201
           1       0.62      0.56      0.59       107

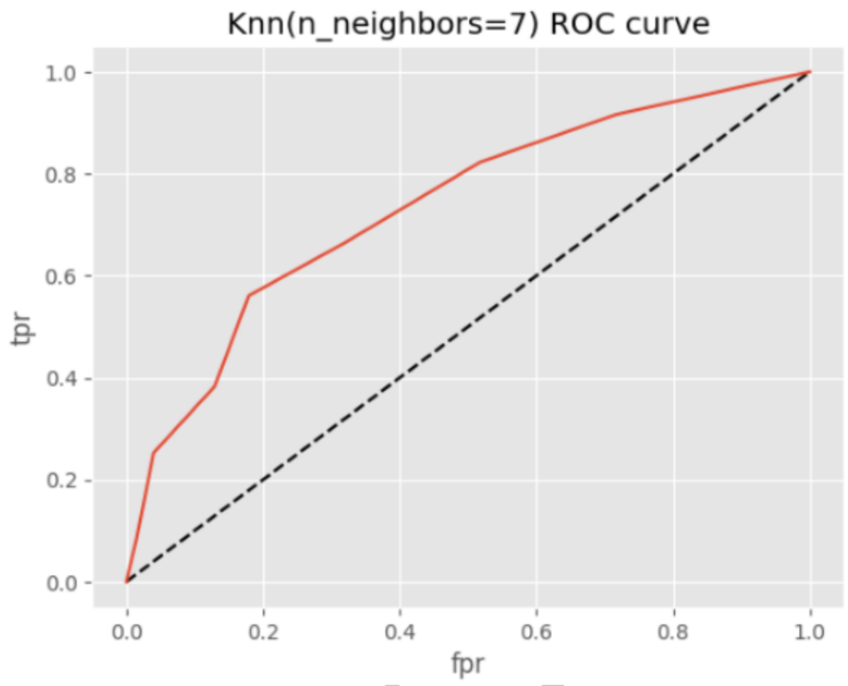
    accuracy                           0.73       308
   macro avg       0.70      0.69      0.70       308
weighted avg       0.73      0.73      0.73       308
```

[13]:
```
from sklearn.metrics import roc_curve
```

[14]:
```
# Predict probabilities for the positive class
y_pred_proba = knn.predict_proba(X_test)[:, 1]
```

[15]:
```
# Calculate ROC curve values
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
```

[16]:
```
# Plot the ROC curve
plt.plot([0, 1], [0, 1], 'k--')
plt.plot(fpr, tpr, label='Knn')
plt.xlabel('fpr')
plt.ylabel('tpr')
plt.title('Knn(n_neighbors=7) ROC curve')
plt.show()
```



Note:-Now, This is an alternative and more **automated** method

[17]:
```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test, y_pred_proba)
```

[17]:
```
0.7345050448691124
```

[18]:
```
from sklearn.model_selection import GridSearchCV
```

[19]:
```
param_grid = {'n_neighbors': np.arange(1, 50)}
```

[20]:
```
knn = KNeighborsClassifier()
knn_cv = GridSearchCV(knn, param_grid, cv=5)
knn_cv.fit(X, y)
```

[20]:

GridSearchCV

best_estimator_:
KNeighborsClassifier

KNeighborsClassifier

[21]:
```
knn_cv.best_score_
```

[21]:
```
0.7578558696205755
```

[22]:
```
knn_cv.best_params_
```

[22]:
```
{'n_neighbors': 14}
```